

Recognising C Code Constructs in Assembly

A visual guide to high-level pattern recognition for malware analysts.

```
int process_data(int input) {  
    int result = 0;  
    int i;  
    for (i = 0; i < input; i++) {  
        if (i % 2 == 0) {  
            result += i * 2;  
        } else {  
            result += i;  
        }  
    }  
    return result;  
}
```

```
0x401000 <process_data>: push    ebp  
0x401001                mov     ebp, esp  
0x401003                sub     esp, 0x10           ; int result, i  
0x401006                mov     DWORD PTR [ebp-0x4], 0x0 ; result = 0  
0x40100D                mov     DWORD PTR [ebp-0x8], 0x0 ; i = 0  
0x401014                jmp     0x40101f           ; jump to loop check  
0x401016 <loop_start>:  
0x401019                mov     eax, DWORD PTR [ebp-0x8] ; i  
0x401019                and     eax, 0x1           ; i % 2  
0x40101C                test    eax, eax  
0x40101E                jne     0x40102f           ; if not even, jump to else  
0x401020                mov     eax, DWORD PTR [ebp-0x8] ; i  
0x401023                shl     eax, 0x1           ; i * 2  
0x401025                add     DWORD PTR [ebp-0x4], eax ; result += ...  
0x401028                jmp     0x401036           ; jump to loop increment  
0x40102A <else_block>:  
0x40102D                mov     eax, DWORD PTR [ebp-0x8] ; i  
0x40102D                add     DWORD PTR [ebp-0x4], eax ; result += i  
0x401030 <loop_inc>:  
0x401034                add     DWORD PTR [ebp-0x8], 0x1 ; i++  
0x401034 <loop_check>:  
0x401037                mov     eax, DWORD PTR [ebp-0x8] ; i  
0x401037                cmp     eax, DWORD PTR [ebp+0x8] ; i < input  
0x40103A                jl      0x401016           ; jump back to loop start  
0x40103C                mov     eax, DWORD PTR [ebp-0x4] ; return result  
0x40103F                leave  
0x401040                ret
```

The Reverse Engineer's Dilemma

The Reality

[illegible]

The Objective

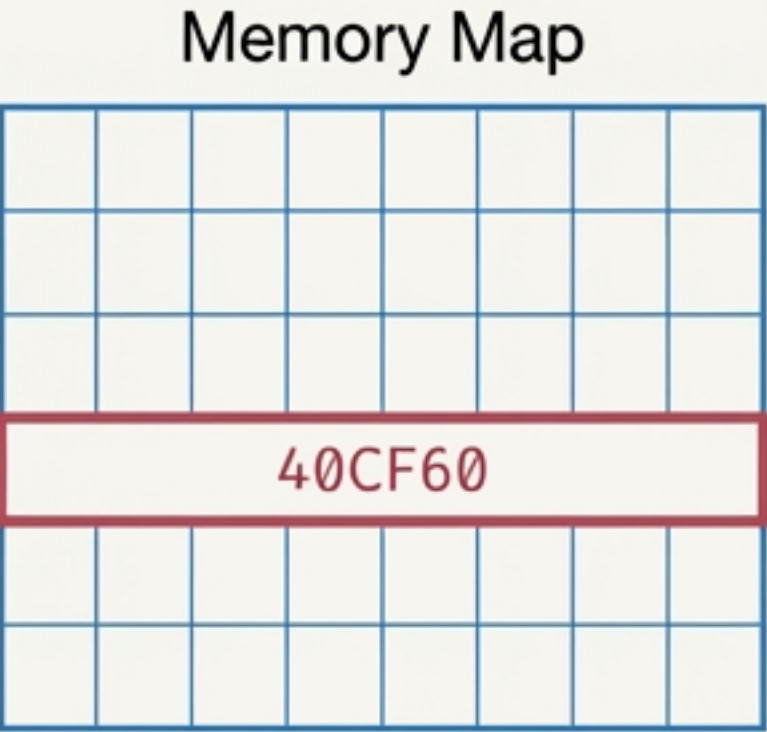
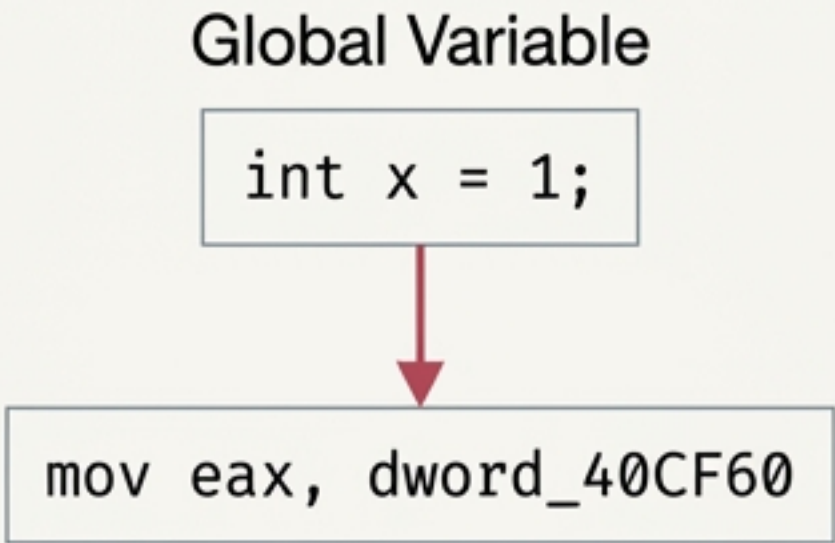
```
void process_data(int* data) {
    if (data != NULL) {
        validate(data);
        compute(data);
    }
}
```

Disassembled programs contain millions of instructions.

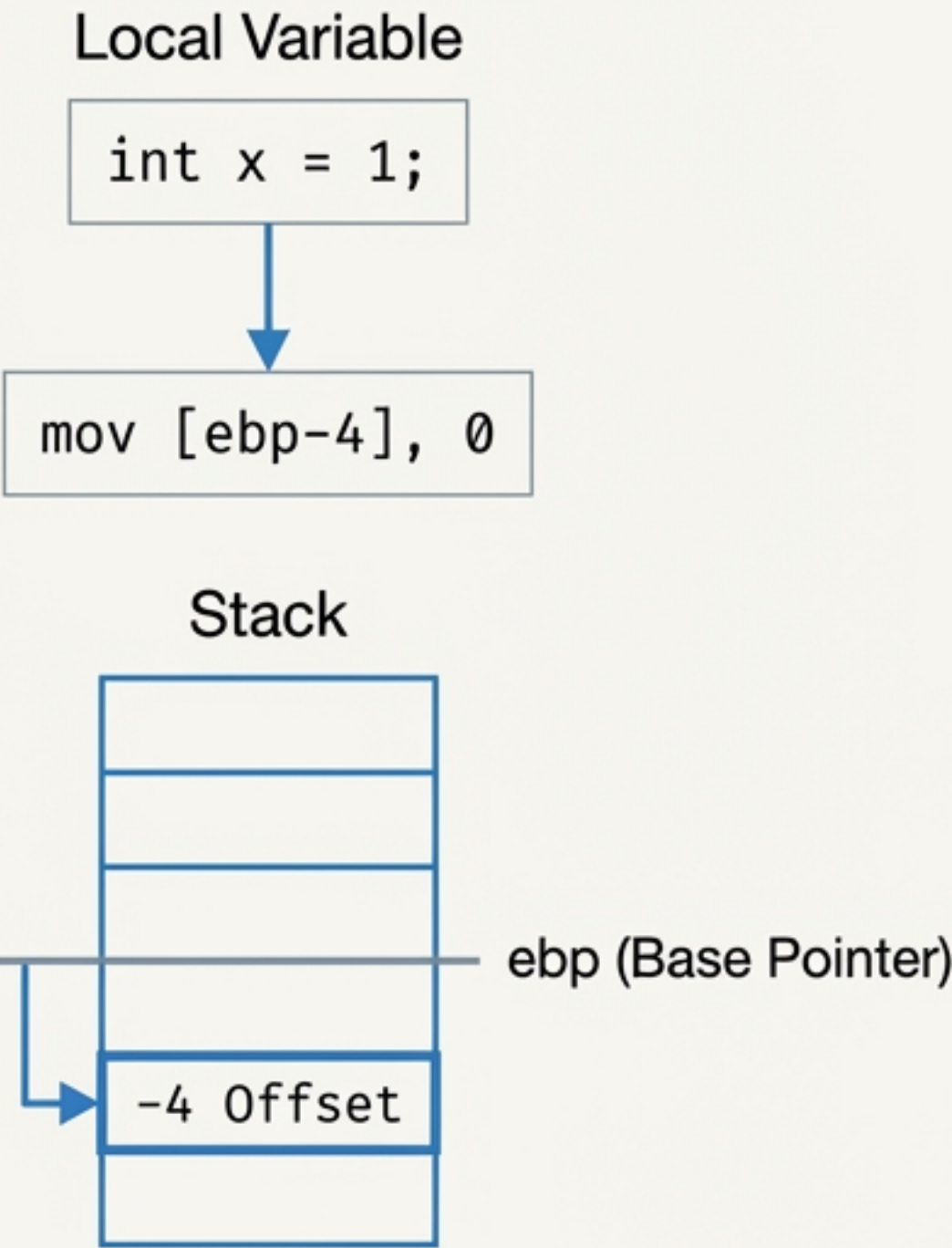
Do not evaluate every instruction individually.

Obtain a high-level picture of functionality by analysing instructions as groups and recognising their structural shapes.

Data Foundations: Global vs. Local Variables

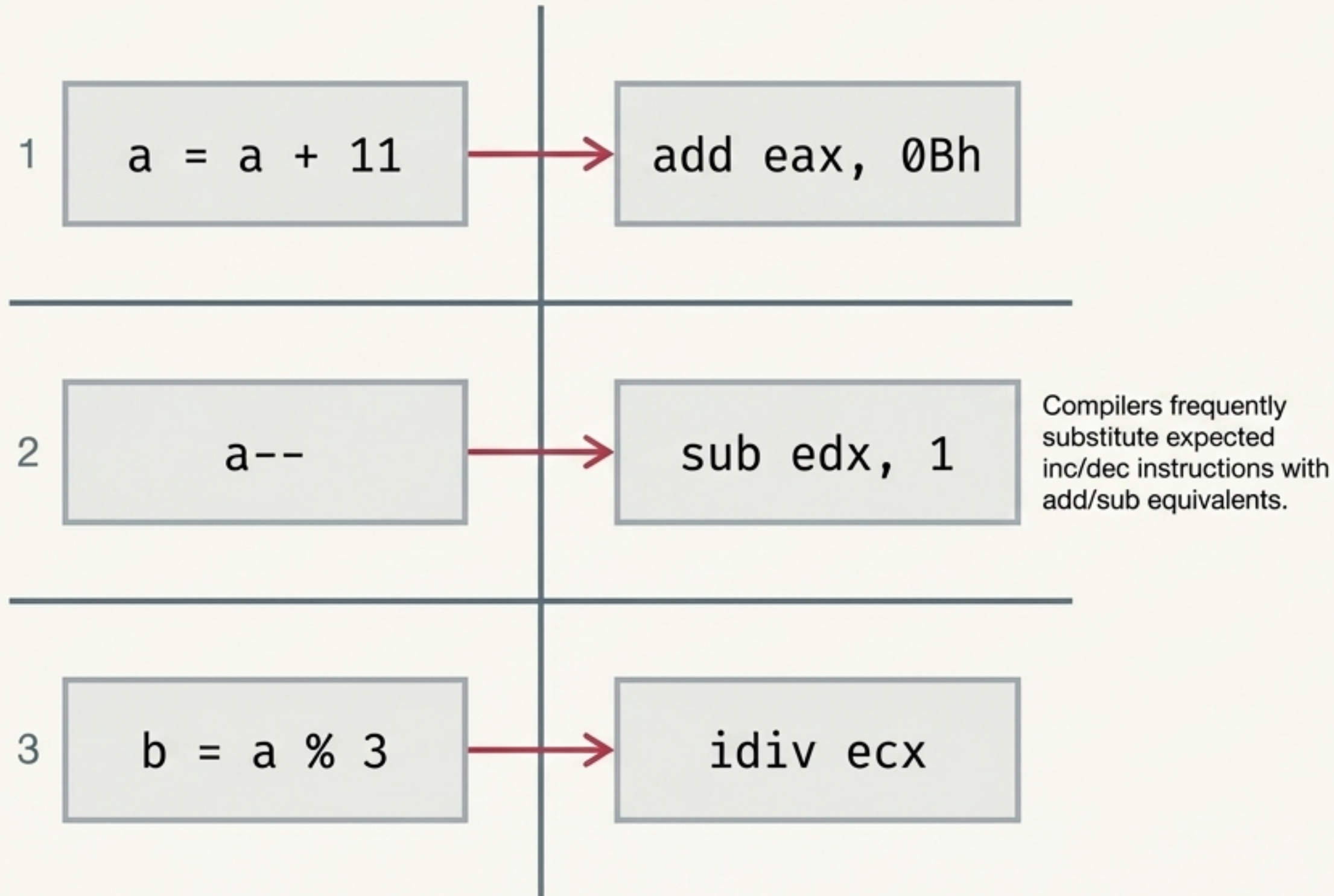


Global: Referenced by hardcoded memory addresses. Accessible anywhere.



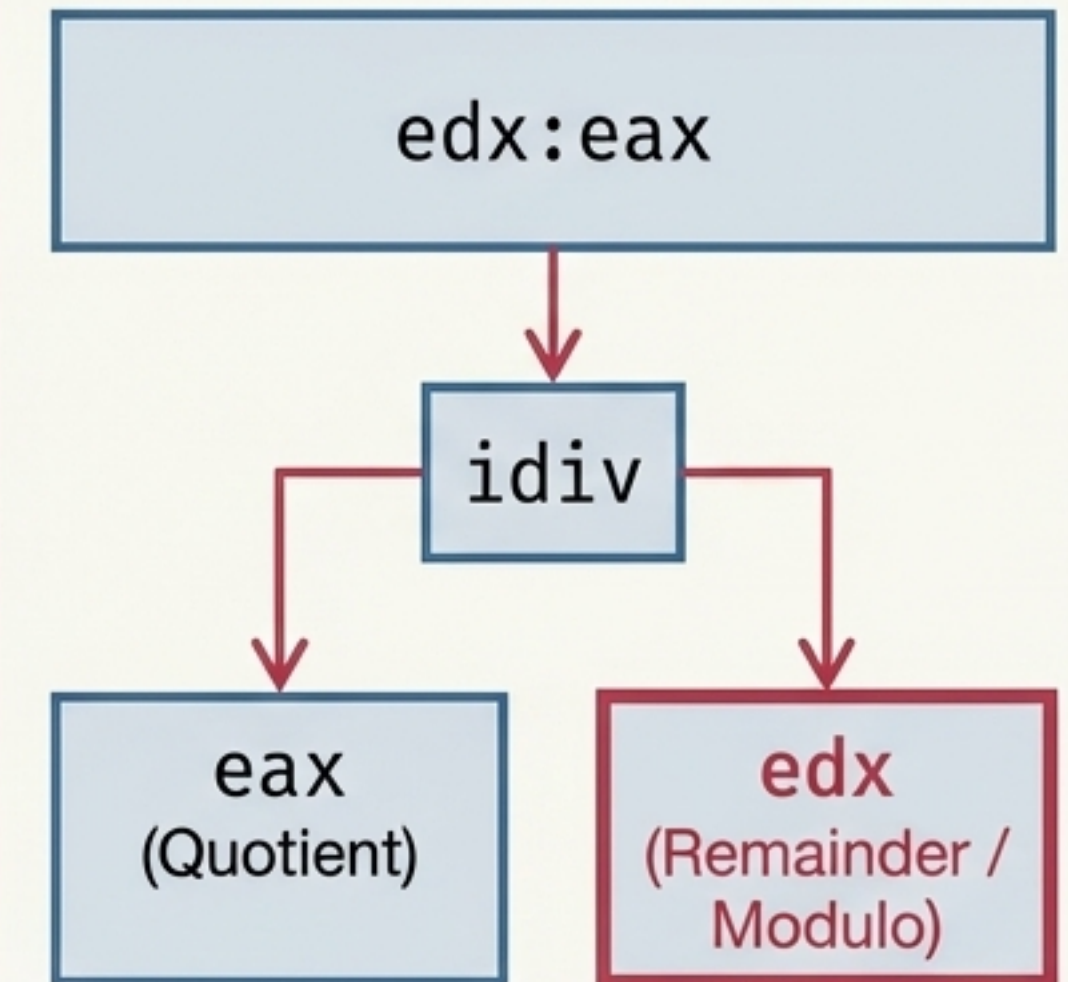
Local: Referenced via temporary offsets relative to the stack base pointer. Accessible only within the function.

The Anatomy of Arithmetic

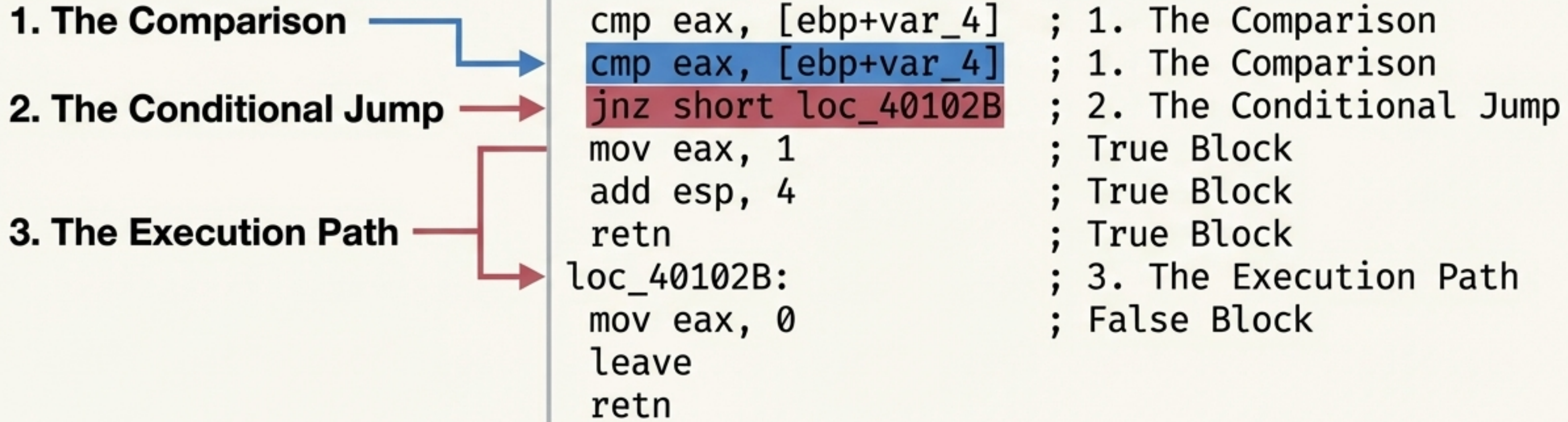


The Modulo Signature

Division and modulo operations utilise the `edx:eax` register pair.



Branching: The Shape of an **if** Statement

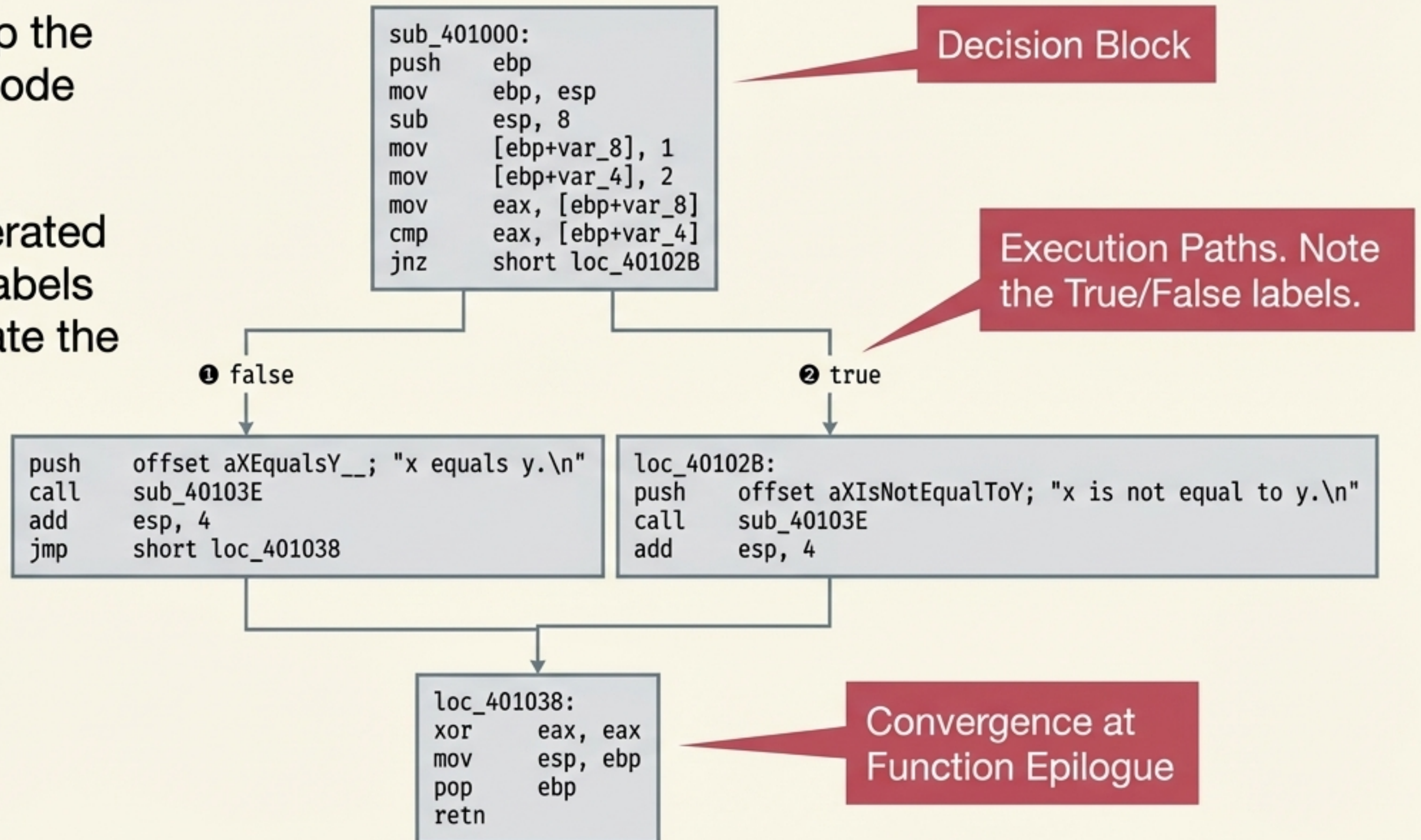


Every if statement requires a conditional jump, but not every jump is an if statement. Recognise the pattern: Compare (**cmp**) -> Jump if Condition is Met/Unmet (**jcc**) -> Execute the resulting path.

Visualising Control Flow (IDA Pro Graphs)

Graphing tools map the physical paths of code execution.

Automatically generated 'True' and 'False' labels drastically accelerate the identification of branching logic without reading every line.

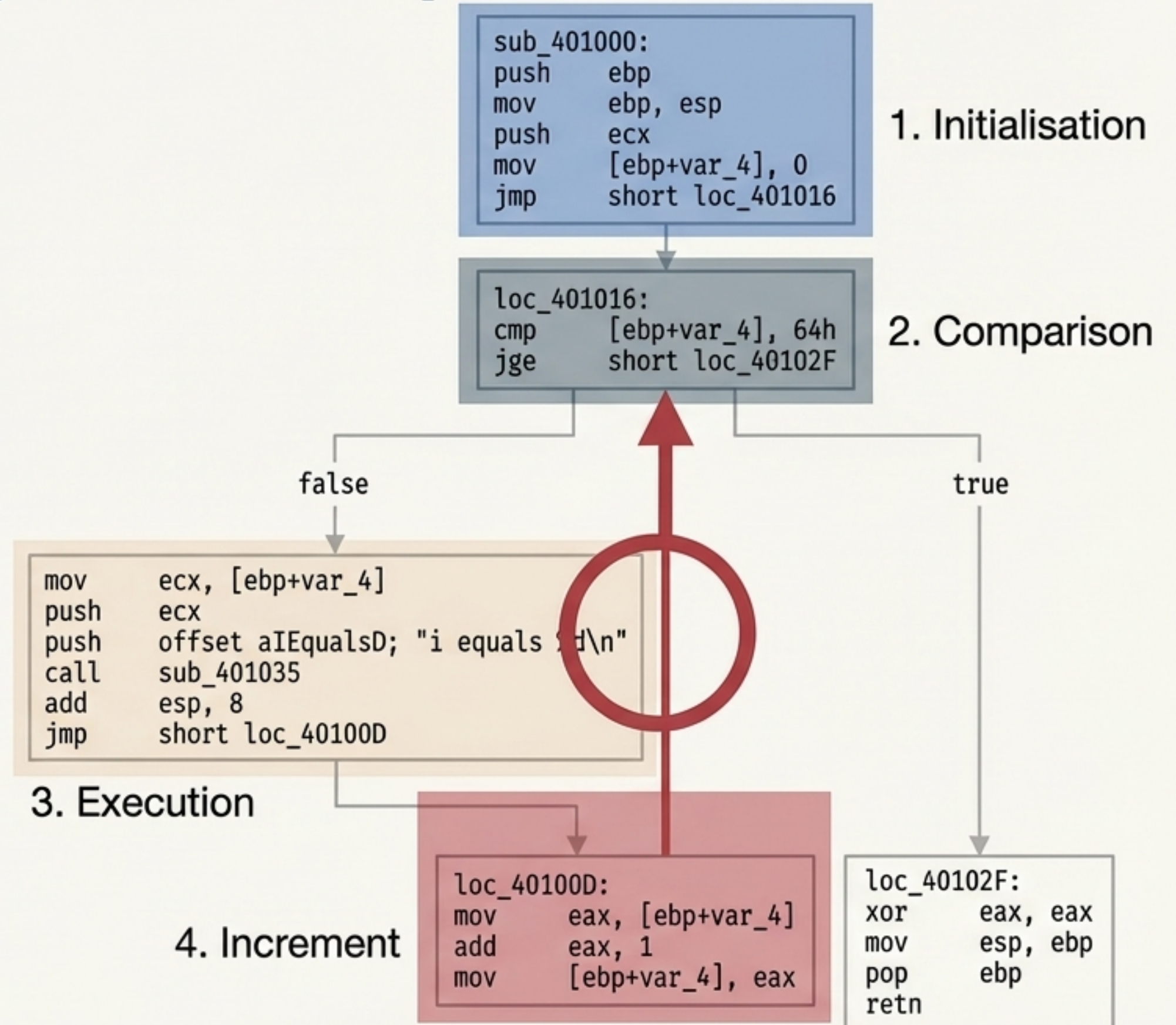


Looping: Deconstructing the for Loop

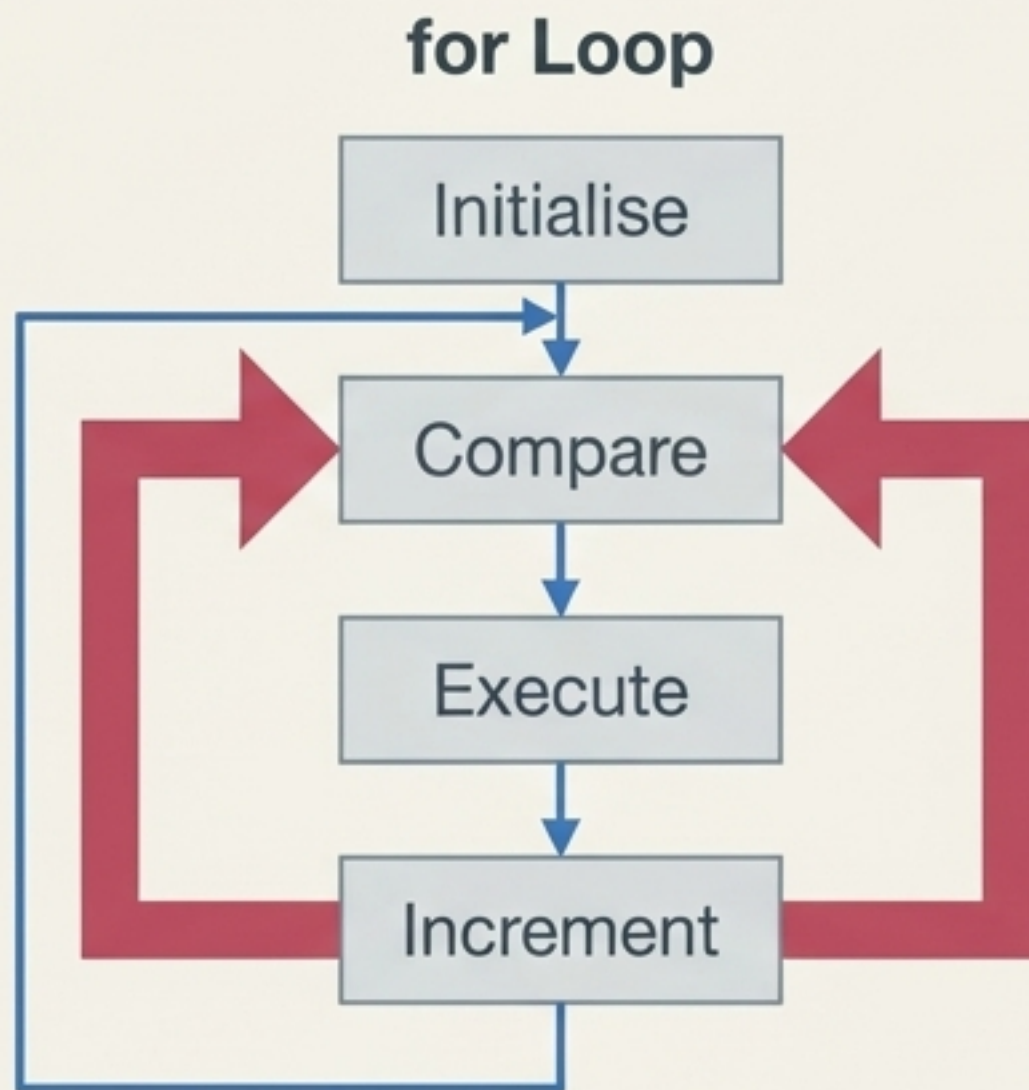
A **for** loop always contains these four distinct components.

The Universal Signature:
Look for the upward-pointing arrow in graph view.

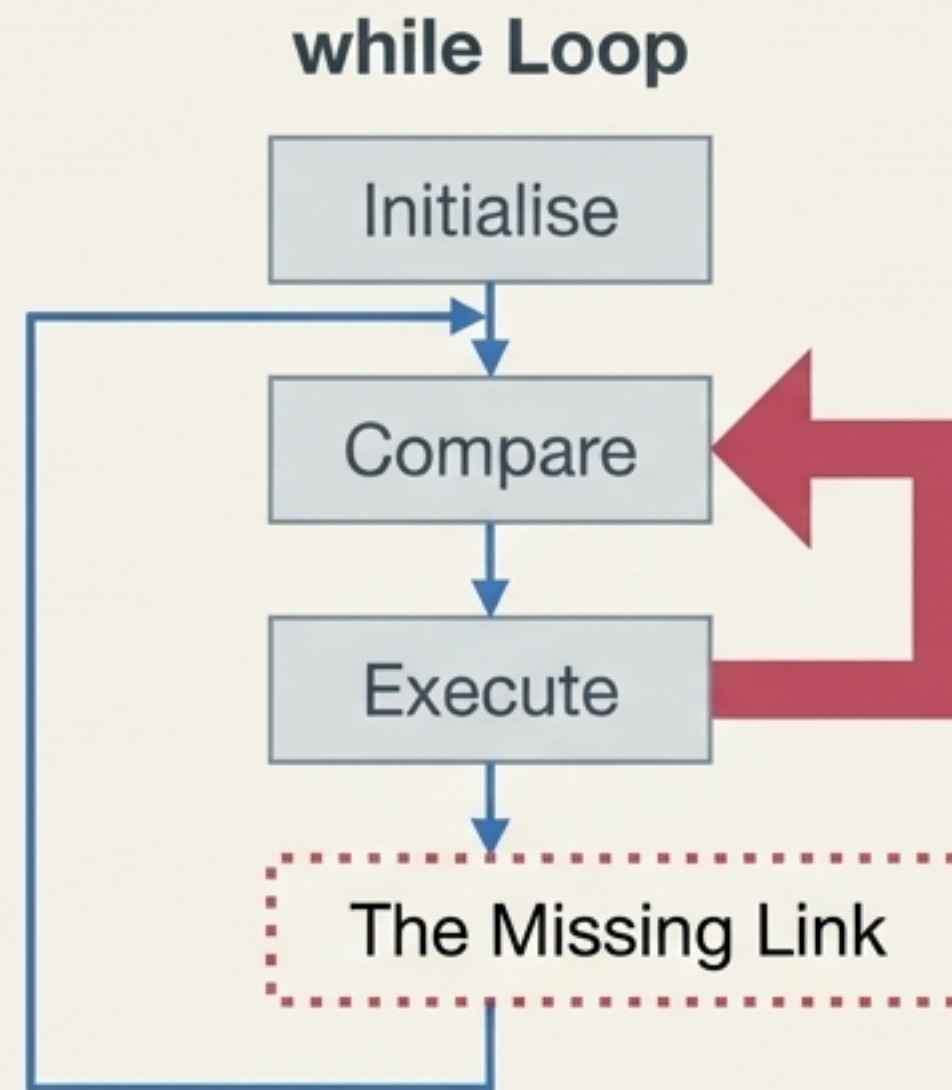
Recognising a jump backwards to a previous memory address instantly identifies a loop.



Looping: The while Loop



Unlike a **for** loop, a **while** loop lacks a distinct increment section.



Malware Application: Commonly used for continuous listening (e.g., waiting endlessly for a packet or command). It relies entirely on its conditional jump (jnz) to eventually break the cycle.

Comparison Matrix: Function Calling Conventions

	cdecl	stdcall	fastcall
Parameter Order	Right-to-Left	Right-to-Left	Right-to-Left (First two in registers)
Stack Cleanup	Caller cleans	Callee cleans	Caller cleans (usually)
Parameter Passing	Pushed to Stack	Pushed to Stack	Registers EDX & ECX, then Stack

- `stdcall` is standard for Windows APIs (the **DLL** handles the stack cleanup).
- `fastcall` is the most efficient, bypassing the stack for the first few arguments by using registers.

Stack Manipulation: Push vs. Move

Visual Studio Method

```
push eax  
push ecx  
call adder  
add esp, 8
```

(Restores the stack pointer)

GCC Method

```
mov [esp+4], eax  
mov [esp], eax  
call adder
```

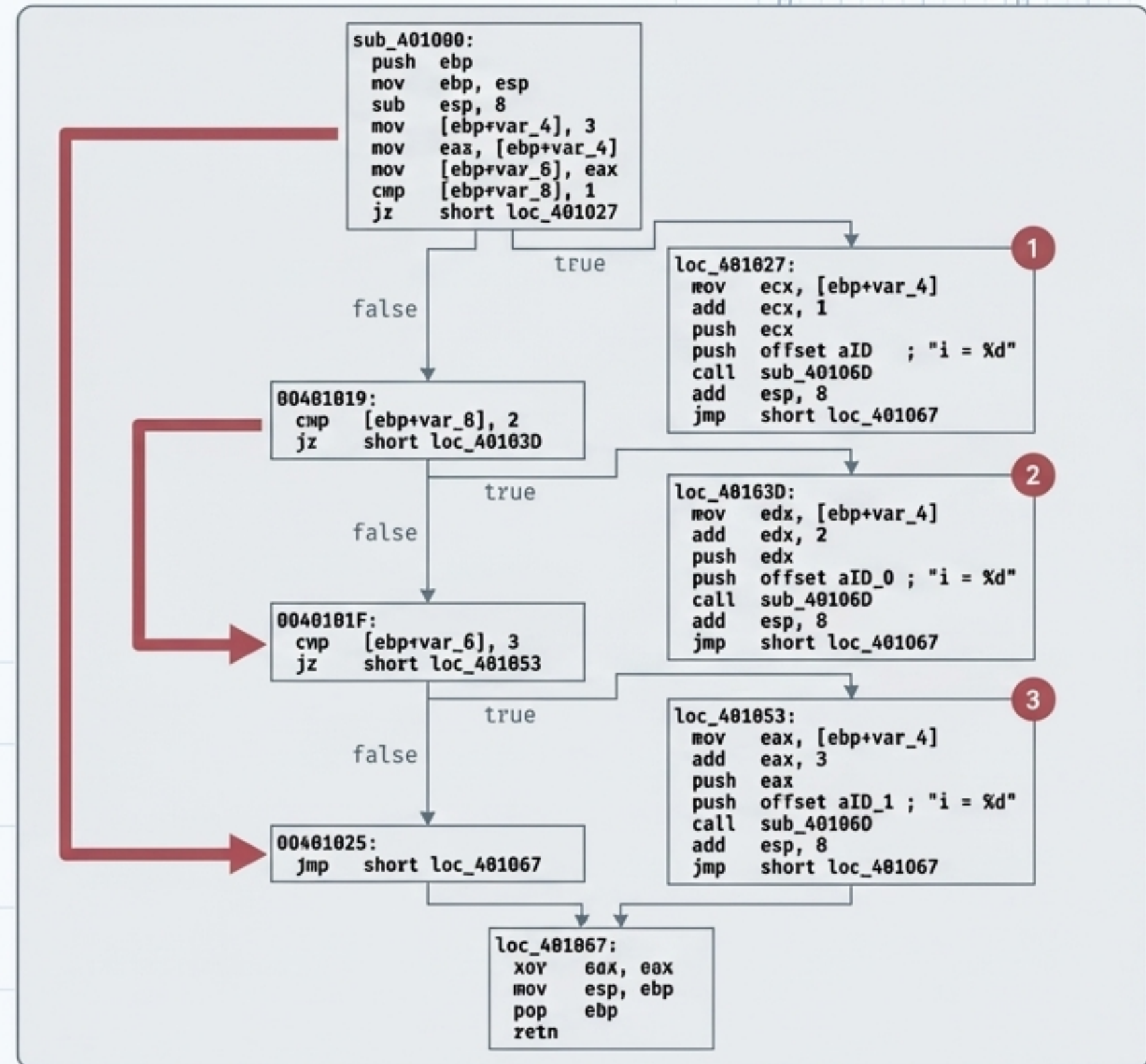
(No stack restoration needed)

Compilers achieve the same parameter passing differently. Some dynamically push directly to the stack, whilst others manually adjust the stack pointer once and mov variables into place. Analysts must recognise both patterns as functionally identical.

Switch Statements: The if Style Cascade

For small contiguous datasets, compilers translate switch statements into a sequential series of `if` statements (`cmp` followed by `jz`).

If a condition is met, the code jumps to a specific block, executes, and then jumps unconditionally to the end of the function. In disassembly, this is structurally indistinguishable from nested `if` statements.

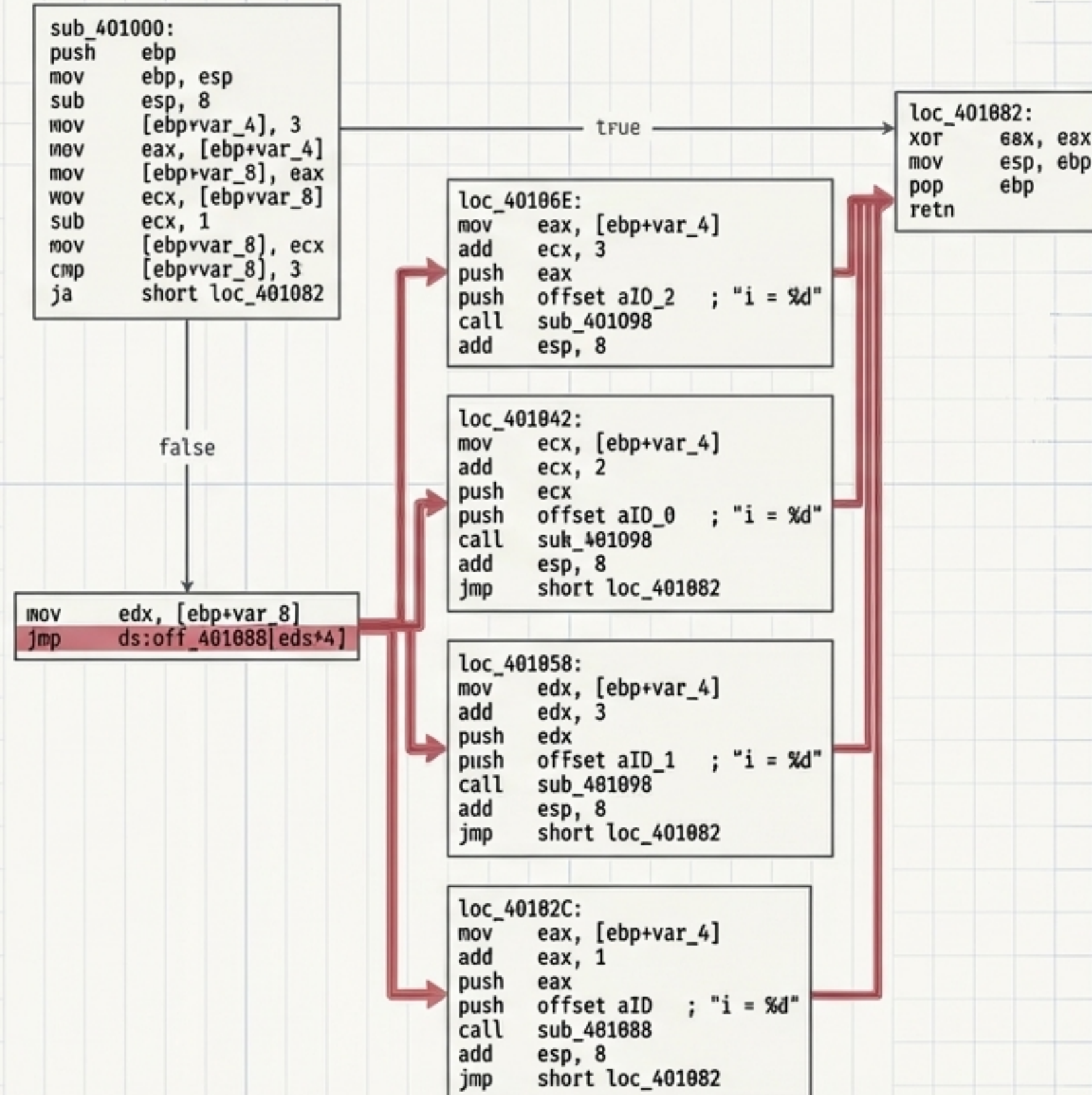


Switch Statements: The Jump Table Optimisation

The Signature

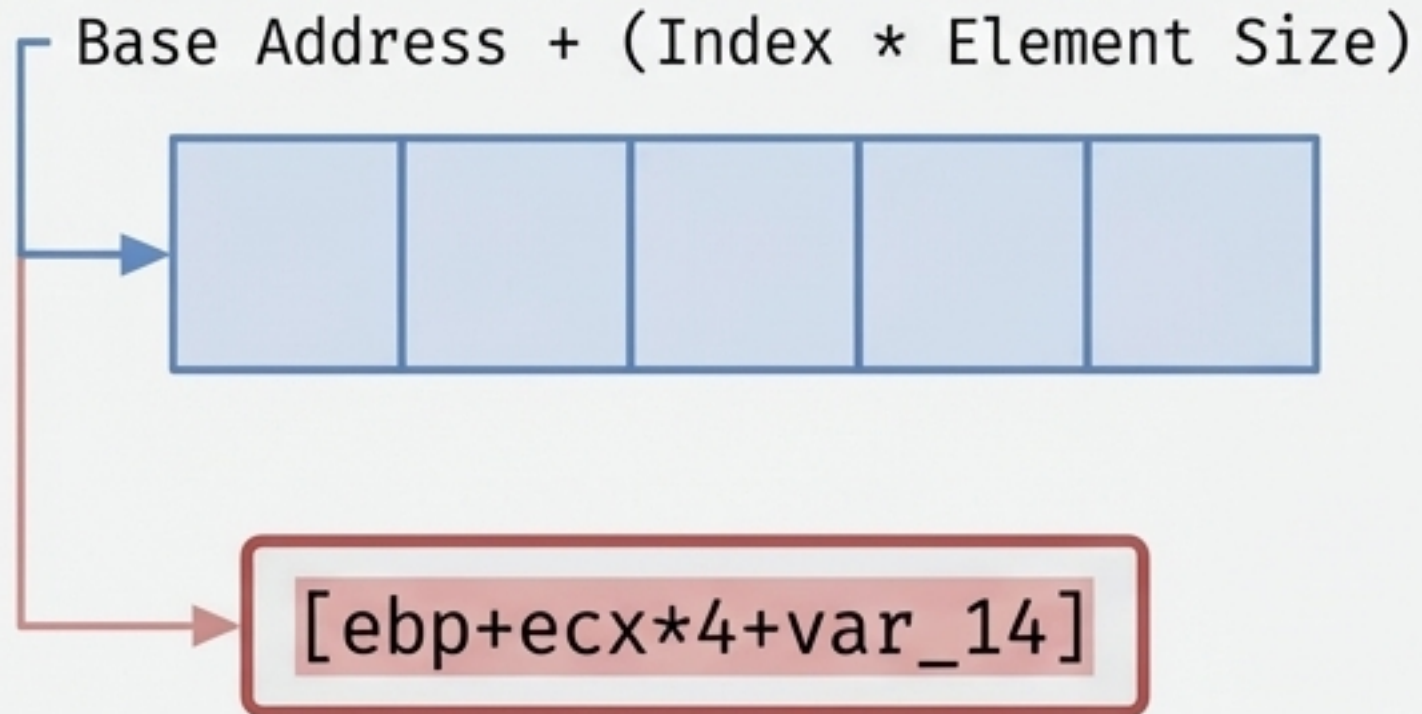
```
jmp ds:off_401088[edx*4]
```

To avoid sequential comparisons in large datasets, compilers use an index to query a jump table. The switch variable is multiplied by 4 (the exact byte size of a memory address) to calculate the precise execution path instantly.



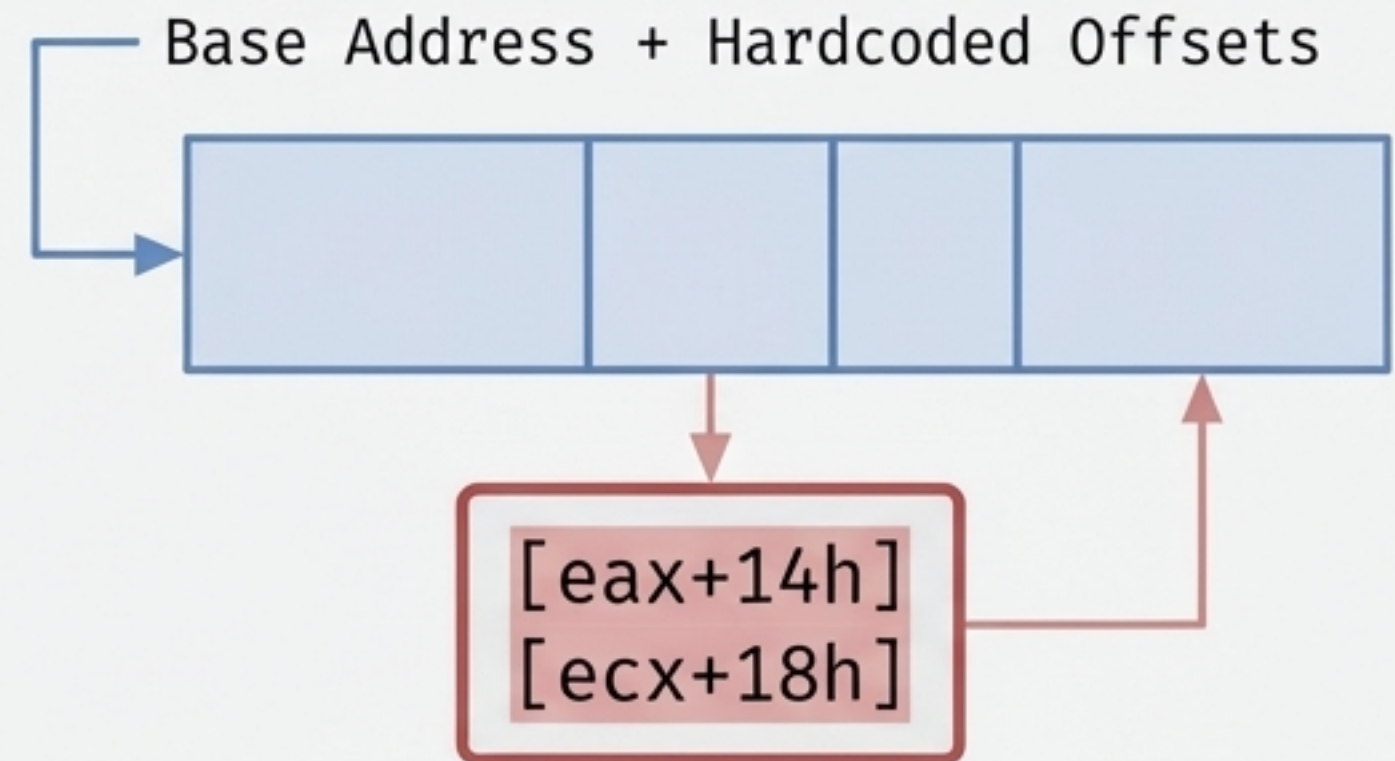
Memory Structures: Arrays vs. Structs

Array



Arrays navigate via a mathematical index multiplier applied to uniform data types.

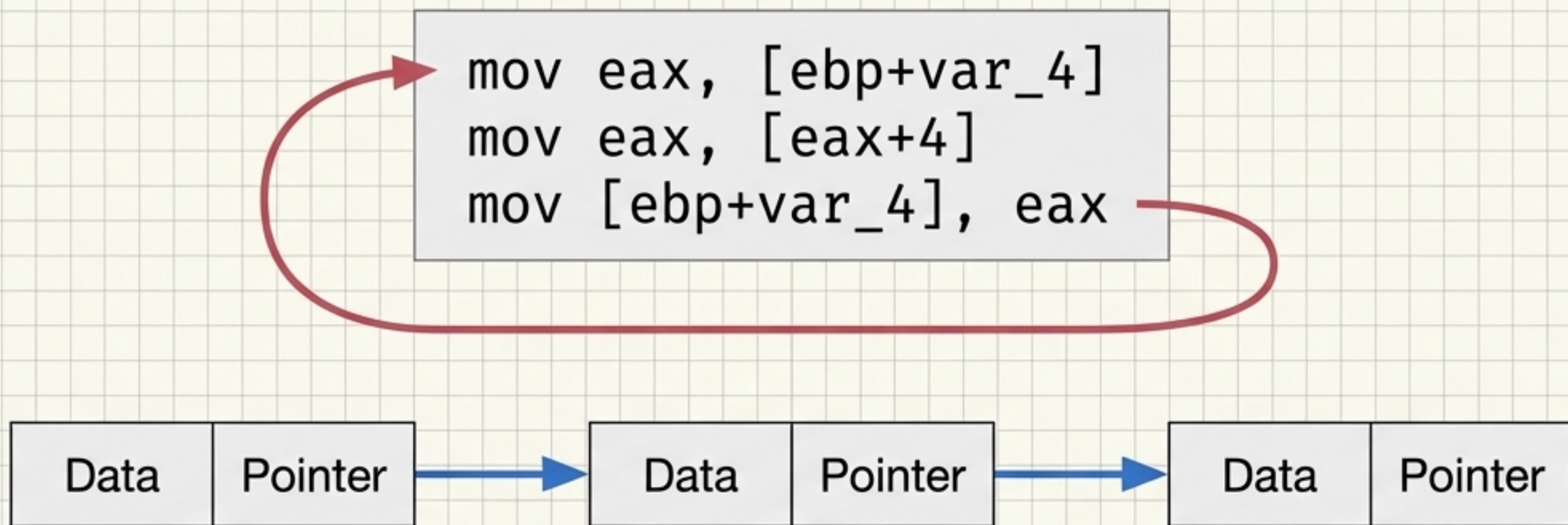
Struct



Structs rely on distinct, hardcoded offsets for differently typed elements.

Pro Tip: Use the 'T' hotkey in IDA Pro to assign struct definitions, turning cryptic offsets (+14h) into readable names (my_structure.y).

Complex Traversal: Linked Lists



The **structural signature of a linked list** is recursive assignment.

When a register is assigned a value that was derived from a specific offset of its previous self (e.g., `[eax+4]`), you are witnessing the traversal of linked nodes in memory.

Synthesis: The Malware Analyst's Cheat Sheet

x86 Assembly Signature	C Structural Equivalent
<code>ebp - offset</code>	Local Variable (Stack)
<code>cmp + jcc forwards</code>	if statement (Branch)
<code>jmp backwards</code>	Loop (for or while)
<code>jmp [base + index*4]</code>	switch statement (Jump Table)
<code>mov eax, [eax + offset]</code>	Linked List (Recursive Traversal)

Abstract yourself from the details. Do not get bogged down in the minutiae. Look for the structural shapes, categorise the construct, and uncover the overarching functionality.